

YOUDESCRIBE ANDROID APP -
AN ACCESSIBILITY TOOL FOR ADDING
AUDIODESCRIPTION TO ONLINE VIDEOS

A Thesis submitted to the faculty of

San Francisco State University

In partial fulfilment of

the requirements for

the Degree

Master of Science

in

Computer Science

by

Cheng Li

San Francisco, California

Dec. 2018

Copyright by

Cheng Li

2018

CERTIFICATION OF APPROVAL

I certify that I have read “YouDescribe Android App – An accessibility tool for adding audio description to online videos” by Cheng Li, and that in my opinion this work meets the criteria for approving a thesis submitted in partial fulfillment of the requirement for the degree Master of Science in Computer Science at San Francisco State University.

Dr. Ilmi Yoon, Ph.D. Professor

Dr. Hao Yue, Ph.D. Assistant Professor

YOUDESCRIBE ANDROID APP –
AN ACCESSIBILITY TOOL FOR ADDING
AUDIODESCRIPTION TO ONLINE VIDEOS

Cheng Li

San Francisco, California

2018

The purpose of this project is to build an Android application for “YouDescribe”. “YouDescribe” is a project founded by Smith-Kettlewell Video Description Research and Development Center (VDRDC) with an objective to enable and encourage sighted volunteers to describe online videos for the benefit of the blind and visually impaired. Their platform was designed to have a backend server and three frontend clients (web, iOS and Android) to facilitate this. Due to the increasing availability of online content through smartphones and the convenience it offers, there is a strong need to make YouDescribe available on mobile phones as well. Given these needs, the main objective of this project is to develop a native Android application for YouDescribe that contribute and play back audio descriptions for online videos (particularly YouTube). We believe that the YouDescribe Android app will not only enable visually challenged users to enjoy videos on their mobile devices, but also encourage volunteers to contribute more in making video content accessible to everyone.

I certify that the Abstract is a correct representation of the content of this thesis.

Chair, Thesis Committee

Date

ACKNOWLEDGEMENTS

Firstly I would love to thank my advisor, Dr. Ilmi Yoon for giving me this great opportunity to work in such a meaningful project. Without her guidance, I would not finish this project. Her encouragement, insight and support throughout this project are vital for the success of this project.

I would also thank Dr. Joshua Miele, the Associate Director of Technology Research and Development, Rehabilitation Engineering Research Center on Low Vision and Blindness of the Smith-Kettlewell Eye Research institute for believing in me and allow me as a part of this amazing team working for YouDescribe. I would appreciate for his warmly help and feedback of my work. I am grateful to Rodrigo Leme de Mello for actively helping me in understanding and building the node.js backend and the AWS dev server.

Meanwhile I'd give my appreciation to Dr. Yue Ting Siu, assistant professor in Program for Visual Impairments in SFU for her first hand inputs from the perspective of an experience describer.

Moreover, I'd like to thank Prof. Hao Yue for his feedback and suggestions on my work and thesis. I learnt a lot from his advises.

Lastly, I'd like to thank my family for their love, support, guidance and encouragement.

Table of Content

List of Figures	VIII
List of Tables	X
1 Introduction.....	1
1.1 Objectives	1
1.2 Roadmap	2
2 Requirement Analysis.....	3
2.1 Use cases for blind and visual impaired users	3
2.1.1 Playback of described videos	3
2.1.2 Searching for videos	4
2.2 Use cases for Describers	4
2.2.1 Recording and uploading videos	4
2.3 Terminology.....	5
2.4 Functional Requirement.....	7
2.4.1 Blind user use cases – Video and audio playback.....	7
2.4.2 Descriptor use cases – Recording audio clips to YouTube videos	7
2.4.3 Descriptor use cases - Login and Registration.....	7
2.4.4 Searching for videos	8
2.5 Non-functional Requirement	8
3 System Design and Architecture.....	9
3.1 Design of the YouDescribe platform	9
3.1.1 Overall architecture	9
3.1.2 Data schema and backend server.....	10
3.2 Design of the YouDescribe Android app.....	14
3.2.1 General Page.....	15
3.2.2 Replay Page	19
3.2.3 Authoring Page	20
4 Implementation	26

4.1	Application Architecture.....	27
4.2	General Page.....	28
4.2.1	Recent Description Sub-Page.....	30
4.2.2	Wish list sub-page and my description sub-page.....	33
4.2.3	Search sub-page.....	33
4.2.4	Account page.....	34
4.3	Replay Page.....	35
4.3.1	Replay of YouTube video.....	35
4.3.2	Coordination of audio clips and YouTube video.....	36
4.4	Authoring page.....	38
4.4.1	The note editing panel.....	42
4.4.2	The existing clip list.....	43
4.4.3	The clip recording panel.....	43
4.4.4	The clip editing panel.....	46
4.5	Accessibility features.....	48
5	Testing.....	51
5.1	Function Testing.....	51
5.2	Accessibility Testing.....	54
6	Future Plan.....	57
6.1	Improvement based on feedbacks.....	57
6.2	Deployment.....	59
	Conclusion.....	60
	Reference.....	61

List of Figures

Figure 3-1 Overall Architecture of YouDescribe Platform	10
Figure 3-2 Data Schema for YouDescribe.....	11
Figure 3-3 General Page of YouDescribe Android App.....	16
Figure 3-4 Recent Description List.....	16
Figure 3-5 Wish List	17
Figure 3-6 My Description.....	18
Figure 3-7 Search Page	18
Figure 3-8 Account Page	19
Figure 3-9 Replay Page.....	19
Figure 3-10 YouDescribe Authoring Page Basic Mode	21
Figure 3-11 YouDescribe Authoring Page Recording Mode	22
Figure 3-12 YouDescribe Authoring Page Editing Mode	23
Figure 3-13 Switch between Basic Mode and Recording Mode	24
Figure 3-14 Switch between Basic Mode and Editing Mode	24
Figure 3-15 Anatomy of the Authoring Page	25
Figure 4-1 Code Structure of YouDescribe Android App.....	27
Figure 4-2 Codes within Modules.....	28
Figure 4-3 Switching with Bottom Navigation View	29
Figure 4-4 Switching with ViewPager.....	30
Figure 4-5 Structure of Recent Described Page.....	31
Figure 4-6 Code Snippet for Infinite Loading List.....	32
Figure 4-7 Linked the Infinite Loading Adapter to Recycler View.....	32
Figure 4-8 Callback Function Binding for Each View Holder	32
Figure 4-9 Callback Function Binding for “Describe” Button to Trigger Authoring Page.....	33
Figure 4-10 Recent Description View Holder and Wish List View Holder in Search Page	34
Figure 4-11 Use Google Sign-In Library to Sign In.....	34

Figure 4-12 Code Snippet for YouTubeFragment Initialization.....	36
Figure 4-13 Strategy to coordinate the replay of audio clips.....	37
Figure 4-14 Code snippet for the Monitoring thread.....	37
Figure 4-15 Callback functions to trigger the looping monitoring thread.....	38
Figure 4-16 Authoring page under different modes	39
Figure 4-17 Create singleton instance for each fragment	40
Figure 4-18 Initial fragments in authoring page	41
Figure 4-19 Fragment replacement for mode switching.....	41
Figure 4-20 Snippet for persisting notes	42
Figure 4-21 Editing button listener: switch the note panel with the clip editing panel	43
Figure 4-22 Code snippet when we start recording	44
Figure 4-23 Code snippet for the timing thread to track the duration of the recording.....	44
Figure 4-24 Code snippet when we stop recording	45
Figure 4-25 Start an UploadFileTask for uploading the local static file and metadata	46
Figure 4-26 Code snippet for asynchronized uploading task.....	46
Figure 4-27 Implementation of the nudging function.....	47
Figure 4-28 Implementation of the type switch button.....	48
Figure 4-29 Implementation of deleting task.....	48
Figure 4-30 Attribute of contentDescription for TalkBack	50

List of Tables

Table 3-1 YouDescribe Backend APIs	12
Table 3-2 YouDescribe Functional Modes	15
Table 4-1 Anatomy of the three working models	39
Table 4-2 Fragment for each authoring page component	40
Table 4-3 TalkBack Gestures.....	49
Table 5-1 Function test cases	52
Table 5-2 Accessibility test cases	55

1 Introduction

1.1 Objectives

Online videos have become the dominating media of digital communication over the recent decade [1]. Given the exponentially growing popularity of smart phones and the accelerating speed of the 4G LTE cellular network, it is much easier now to access and upload online videos through smart phones [2]. Online videos, which are not only related to personal life and entertainment, but also often used to convey informative and instructive information, have become the most important media in the era of Internet and social networks [3]. However, not everyone has that privilege to enjoy the online videos. The blind or virtually impaired people, who can only access the audible information of videos, will miss out the visual information and often consequently lose the most part of the information in those videos. An effective approach to bridge this gap is to convert the visual information into the audible type. Technically, it is to attach descriptive audio narrations onto the videos and play this narrative audio tracks along with the original videos [6]. This approach has been widely used in television programs and museum exhibits, and has made visually impaired people engage with the video content to a great extent [7].

YouDescribe is a tool for the benefit of blind or visually impaired to help them enjoy the online videos [8]. It is a free tool that is initialized and developed by the Smith-Kettlewell Video Description Research [9] and Development Center, with funding provided by the U.S. Department of Education, OSEP.

YouDescribe applied the approach introduced before. It follows a crowdsourcing model [10] which encourages sighted volunteers to record and upload audio descriptions to videos upon request. For blind or virtually impaired people, YouDescribe provides a coordinative replay of the original video and the audio description, so to make them enjoy those audio annotated videos.

YouDescribe is now available on the iOS platform and the web browsers. Since the Android system takes 86.2% of the current smart phone market [11], it is necessary to enable the services of YouDescribe for users of Android phones. The current web application of YouDescribe is not applicable on the phones since the web browser on the phone is not suitable for those complex interactions between users and the tool. Meanwhile replay of a video on the web browser client of YouDescribe leverages flash-based API [10],

while flash is not available any longer on the Android platform. Given these two reasons, we need to develop a native Android app as the client of YouDescribe to provide playing back and recording audio descriptions for online videos (particularly YouTube).

This project is to create such a native Android app of YouDescribe, with which visually impaired people can replay YouTube videos with audio annotations, and authors can add audio clips to the videos upon request. We first implemented a video player module, which enables visually impaired users to listen to videos with audio annotation while they are away from the computers. Following this, we implemented the authoring mode, which allows sighted users to record and upload audio descriptions for YouTube videos on an Android phone. Thus, the YouDescribe Android app recreates the web experience for video and audio playback, recording, searching, sharing and requesting for audio descriptions on YouTube videos with full support for low-vision accessibility. The YouDescribe Android app will not only enable visually challenged users to enjoy videos on Android phones, but also encourage volunteers to contribute more in making video content accessible to everyone.

1.2 Roadmap

This report focuses on the design and development of the YouDescribe Android app. We started with introducing “YouDescribe”, as well as the objectives of this thesis. In Chapter 2, we conduct requirement analysis for the design of the YouDescribe Android app. Chapter 3 focuses on the design of YouDescribe Android app, including the overall architecture of the Frontend-backend architecture as well as the UI design of each page. In Chapter 4, we dive into the implementation of each page, and give code snippets of the solution for some important problems and features, especially the solutions for the accessibility related features. Chapter 5 shows the testing of the application, including function testing and usability testing. Finally, we conclude by outlining the deployment strategy and future plans.

2 Requirement Analysis

Requirement analysis is vital in software development, since it transforms high level concepts and requirements of the application into well-established, measureable and testable specifications that have been approved by stakeholders [12]. In this chapter, we will conduct a series of requirement analysis as the first step of the development. We first describe the common use cases of YouDescribe Android app. Then from these use cases, we summarize its desired functional and non-functional requirements.

2.1 Use cases for blind and visual impaired users

2.1.1 Playback of described videos

- a) Alex, a visually impaired person, is travelling home from work on a bus and is connected to the Internet via 4G LTE network with his Nexus 5 (Android 7 - Nougat). He is navigating this Android phone using TalkBack (TalkBack is the Google screen reader included on Android devices. TalkBack gives you spoken feedback so that you can use your device without looking at the screen.). He opens the YouDescribe Android app on his phone, and finds a list of most recently described YouTube videos. He taps on one of them to open and launch the playback of the YouTube video along with the corresponding YouDescribe audio descriptions. He hears the audio descriptions being played back at specific time points during the video playback. Alex is able to understand the visual contents of the video much better through these audio descriptions.
- b) After launching the video in the YouDescribe player, Alex wants to check if there is any other describer who has described the same video. He can select his preferred audio description. He can select another audio description from the drop down list named “Described by”. The same video is then played with another audio description.
- c) Alex can adjust the volume of the audio clips by using the volume control bar on the UI or simply using the physical volume up/down button on the phone.

2.1.2 Searching for videos

- a) Alex is a visually impaired user. His friend told him that there is a video related to “Yellow” that is funny on YouDescribe. He opens YouDescribe on his phone and navigates to the search page. He inputs “Yellow” as the key word in the search bar and some videos related to “Yellow” appear on the result list. He chooses the first one and opens the playback window to enjoy the first video related to “Yellow”.
- b) Tommy is a sighted volunteer. He wants to add some audio descriptions to a movie trailer of an upcoming movie to make it available on YouDescribe. He searches for the name of the trailer on the search page of YouDescribe Android app using his Nexus 5 phone. The search results contain the trailer as a video without descriptions, which has “Describe” button on the result.

2.2 Use cases for Describers

2.2.1 Recording and uploading videos

- a) Tommy is a sighted volunteer, who wants to add some audio descriptions to a movie trailer of an upcoming movie to make it available on YouDescribe. He searches for the name of the trailer on the search page of YouDescribe Android app using his Nexus 5 phone. Upon logging into his account, he enters the authoring page by tapping the “Describe” button of that trailer on the search results.
- b) In the authoring page, Tommy first writes down some textual notes on the note section as the auxiliary notes for the audio descriptions.
- c) Then, he starts the replay of the original video by tapping the play button of the YouTube video player. He taps the big “Plus” button to open the recoding panel of the authoring page. He starts the playback of the original YouTube video and at some specific point during the playback, he taps the “Record” (with microphone icon) button to start recording the descriptions. Tommy taps the “Record” button again to stop the recording.

- d) Tommy adjusts the starting time a bit with the nudge buttons on the recording panel. Then he switches the type of current audio clip from “extended” to “inline” by toggling the “clip type” toggle-switch.
- e) He taps the save button (disk icon) on the panel, and there is a new clip added to the existing audio clip list.

2.3 Terminology

This section describes the common terminologies used in this report.

- a) Online videos: Currently for YouDescribe application, YouTube videos are the only supported source of online videos.
- b) User: Either a sighted or visually impaired person who uses YouDescribe application but may not be an authenticated user.
- c) Descriptor: A sighted volunteer (authenticated through Google log in) who can record audio descriptions for an online video.
- d) Audio descriptions: A set of audio clips played back along with a video describe the visual content of the video to visually impaired users. In the case of YouDescribe, audio descriptions are recorded by volunteers for online videos, which have synchronization and start time metadata stored on the YouDescribe server.
- e) Described videos: Online videos for which audio descriptions are recorded by a YouDescribe descriptor.
- f) Non-described videos: Online videos for which there are no recorded audio descriptions by a YouDescribe descriptor.
- g) Inline audio description: An audio clip that is played over the video when the video is playing. The playback time remains intact with the use of inline audio descriptions.

- h) Extended audio description: An audio clip that is played over the video when the video is paused. The playback time effectively increases with the use of inline audio descriptions.
- i) Player: The YouDescribe player that can play video content along with the recorded audio descriptions.

2.4 Functional Requirement

2.4.1 Blind user use cases – Video and audio playback

- a) The user (sighted or visually impaired) should be able to play a listed YouTube video along with its associated inline and extended audio descriptions at appropriate designated start times.
- b) The user should be able to choose the describer from the available describers for the video s/he is interested in. The default describer will currently be the author associated with the video listing.
- c) The video player selected for the app must be able to intercept playback events at a very high rate (accuracy) to enable the audio clip to play at the intended time during video playback.

2.4.2 Describer use cases – Recording audio clips to YouTube videos

- a) The recorder should allow an authenticated user to record audio descriptions and upload them to the server.
- b) It should allow the user to preview their audio clips over the YouTube video in a player prior to uploading the clips.
- c) It should allow the describer to edit properties and delete one or more audio clips belonging to the describer.

2.4.3 Describer use cases - Login and Registration

- a) Google authentication will be supported
- b) Without logging in, a user is able to browse the website and perform all functions except for recording audio descriptions.

2.4.4 Searching for videos

- a) The home page should display a list of described videos that are ordered by date of creation.
- b) The user should be able to search for videos by video name.
- c) When a user search for a keyword, it should display a list of described videos matching that query.
This list will be followed by a list of YouTube videos matching that query.

2.5 Non-functional Requirement

- a) The application will be developed in Android Studio using Java.
- b) The application will be made accessible according to well-known Android standards and guidelines for human interface and accessibility.
- c) The application will be optimized for display on all known screen sizes of android devices.
- d) The application will be easy to use and should require minimum training.
- e) The application will follow security standards.

3 System Design and Architecture

3.1 Design of the YouDescribe platform

3.1.1 Overall architecture

YouDescribe platform is designed to provide services on the platforms of Web browser, iOS system and Android system [13]. YouDescribe applies the “Frontend – Backend” pattern to modularize the logic and data flows within different services.

To put things simple, YouDescribe replays the original YouTube video along with those descriptive audio clips coordinately so that visually impair people can understand the video by hearing the two sound tracks. Figure 3-1 demonstrates the overall architecture of the YouDescribe. Three types of frontend clients serve as the user interfaces to interact directly with users. When replaying, the frontend clients fetch data from YouTube API to replay the original YouTube video. Meanwhile, the clients also fetch metadata and static audio clip files of the YouDescribe related to that YouTube video from the YouDescribe API to replay the descriptive audio clips along with the YouTube video.

We create a universal backend with NodeJS and MongoDB [14 - 16] to serve the metadata and statics files of the audio clips for all the three clients. Each client is responsible for fetching data and files, as well as replaying all the related videos and audio tracks properly to fulfill the user requirements.



Figure 3-1 Overall Architecture of YouDescribe Platform

3.1.2 Data schema and backend server

The metadata of each audio clip is vital for the coordinative replay of the two sound tracks. We design a data schema to host the information of all the audio clips related to one specific YouTube video.

We create a two-hierarchical data schema to connect a YouTube video with all its related audio clips. Figure 3-2 displays the data schema for YouTube video with YouTube id of “IHUo4BpSinM”. In this schema, audio_description is to represent a complete series of descriptive audio clips for one YouTube video. In every audio_description, there are multiple audio_clips which represent the descriptive sound tracks. One YouTube video can have multiple audio_descriptions since there might be multiple contributors. Every contributor can create an audio description independently. For the case in Figure 3-2, the YouTube video “IHUo4BpSinM” has two audio descriptions. The first audio description with id of “5bd2a71db55a005b89021526” has 6 audio clips attached, among which the first one is “5bd2abadf5f7735b4e876df7”.

For audio_clips, the most important metadata are “play_back_type”, “start_time”, “end_time”, “file_name” and “file_path”. Users can readily tell their purpose of use from their names. The metadata is transferred via JSON data from the backend to the frontend. The metadata help frontend client retrieve the static audio files, and control the replay of the sound track during the replay of the YouTube video.

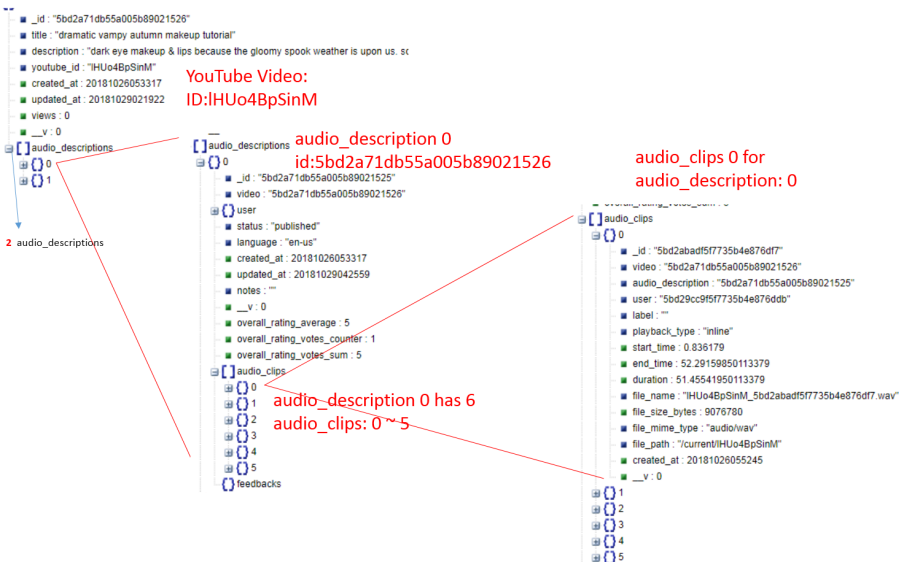


Figure 3-2 Data Schema for YouDescribe

Metadata is hosted in the universal backend server. Retrieving data from that server needs to use http “GET” method along with relevant parameters. Creating new audio clips will trigger a “POST” http request with all new information wrapped in JSON [17] data as the payload. Updating existing audio clips can use “PUT” method with necessary information wrapped in JSON data. All responses from the backend server are wrapped in the JSON data. Details of the backend APIs are listed in Table 3-1. More details of the Backend API could be found at the Github [18] repo:
<https://github.com/SmithKettlewellRERC/YouDescribe-api>

Table 3-1 YouDescribe Backend APIs

Function	Detail
User Authentication	<code>/auth</code> POST <code>/auth</code> body json raw googleToken
User Information	<code>/users</code> GET (Retrieve an user) <code>/user/:userId</code>
Wish list Related Operations	<code>/wishlist</code> POST (add video to wish list) <code>/wishlist</code> body json raw youtubeId GET (get all items from wish list) <code>/wishlist</code> GET (get one item from wish list) <code>/wishlist/:youtubeId</code> body json raw youtubeId PUT (update wish list item) <code>/wishlist/:youtubeId</code>
YouTube Video Related Operations	<code>/videos</code> GET (get all videos) <code>/videos</code> GET (search) <code>/videos/search</code> GET (get on) <code>/videos/:id</code> GET (get videos by userId) <code>/user/:userId</code>

Audio Description Related Operations	<pre> /audiodescriptions POST (To publish or unpublish an audio description) /audiodescriptions/:audioDescriptionId body json raw userId userToken action ('publish' 'unpublish') PUT (To update notes of audio descriptions) /audiodescriptions/:audioDescriptionId body json raw userId userToken notes </pre>
Audio Clips Related Operations	<pre> /audioclips POST (upload) /audioclips/:videoId body json raw userId userToken audioDescriptionId audioDescriptionNotes title description label playbackType startTime endTime duration file.size file.mimetype DELETE (delete) /audioclips/:audioClipId body json raw userId userToken PUT (update) /audioclips/:audioClipId body json raw userId userToken label </pre>

3.2 Design of the YouDescribe Android app

The YouDescribe Android app serves as the interface where users interact with YouDescribe on Android phones. We analyze the use cases listed in Chapter 2, design three modes that the YouDescribe Android app can work in for these use cases.

Table 3-2 YouDescribe Functional Modes

Mode	Description	Use Cases
General Mode	Serves as the front page, provide browsing, searching, wish list operations, user login	Browsing Existing Browsing WishList Search for Videos
Replay Mode	Replay YouTube video along with the descriptive audio clips	
Authoring Mode	Create new Audio Description or Add new Audio Clips to an existing Audio Description.	

We design a function page for each function mode. Features related to that function mode are all accommodated in that page.

3.2.1 General Page

The general page is the entrance of YouDescribe Android app. It consists of 5 sub-pages, each of which fulfills a specific use case (Figure 3-3). The 5 sub-pages are accommodated by a bottom-navigation bar. User can easily tab the icon on the bar to switch between different pages. Swiping left / right can also lead user to the previous / next sub-page of the current sub-page.

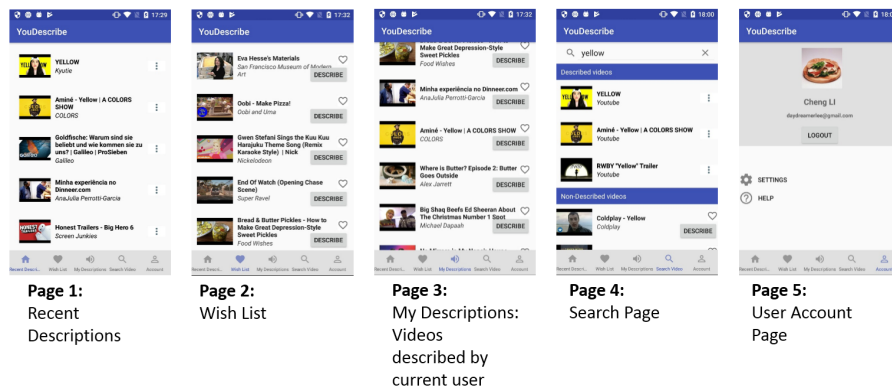


Figure 3-3 General Page of YouDescribe Android App

Sub-page 1 is the portal where users (most likely the visual impaired people) can see a specific YouTube video with its audio descriptions. This page is designed to hold an infinite list where every item on this list is touchable. The list is scrollable. Touching an item on this list will lead the user to the replay page of the YouTube video for that item.

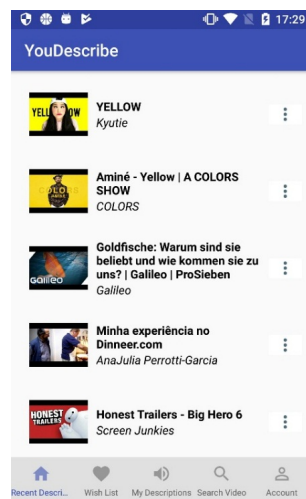


Figure 3-4 Recent Description List

Sub-page 2 (Figure 3-5) works as the wish list for videos to be described. It is also an infinite list, where tapping the “Describe” button for each item will lead the author to the authoring page for that video. People can create / update audio descriptions for that video in the authoring page.

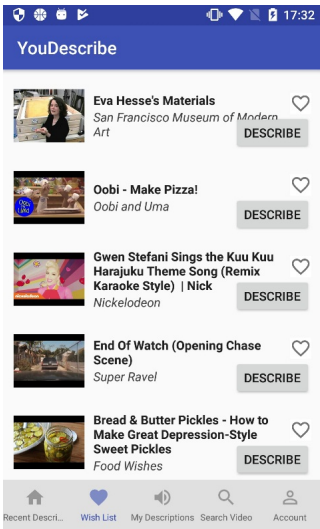


Figure 3-5 Wish List

Sub-page 3 (Figure 3-6) lists all the videos that have been described by the current signed-in user. User, as an author, can tap the “Describe” button to edit and update the existing audio descriptions contributed by the current user.

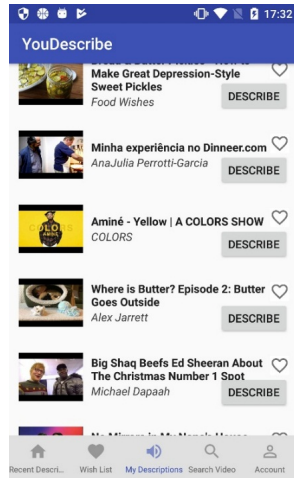


Figure 3-6 My Description

Sub-page 4 (Figure 3-7) is the page for searching. After a user inputs the keywords in the search bar, the app will return the described and non-described videos to the user.

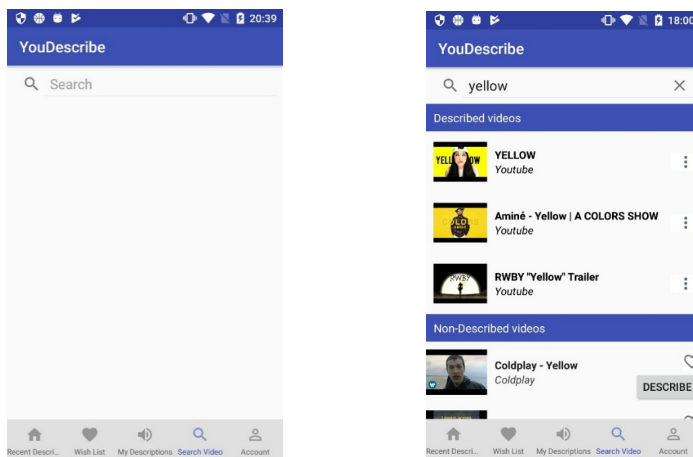


Figure 3-7 Search Page

Sub-page 5 (Figure 3-8) is the user account page. User needs to sign in to YouDescribe in this page before creating or editing audio descriptions.

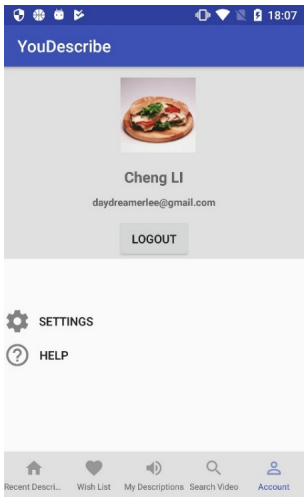


Figure 3-8 Account Page

3.2.2 Replay Page

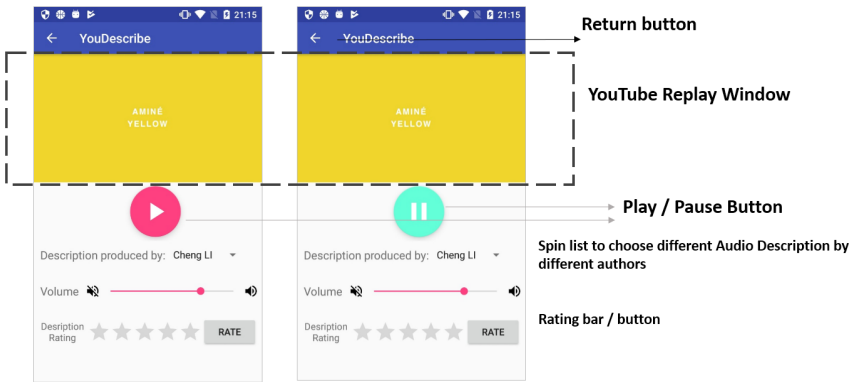


Figure 3-9 Replay Page

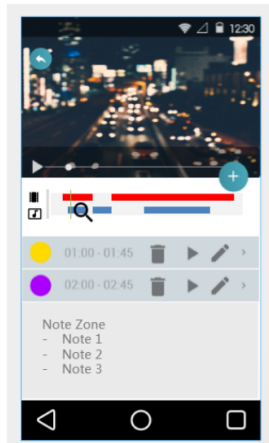
3.2.3 Authoring Page

The authoring page is designed to serve as the all-in-one center where an author can create / update audio descriptions for a specific YouTube video. According to the use cases, the authoring page has two major functionalities:

1. Recording a new clip (under this scenario, users need to see the video as well as some textual notes to finish a recording process, but do not need to see the existing clip list)
2. Editing / modifying an existing clip (under this scenario, users need to see the video, probably see other clips. Since the record is done, users just need to nudge the beginning and ending time, but do not need to see the textual notes anymore)

Given the above analysis, we propose the authoring page under three modes: basic mode, recording mode and editing mode.

Authoring page basic mode is the mode where an author will see when he enters the authoring page. The functions it provides include “adding new textual notes to assist further recording” and “browsing the existing audio clips”. Figure 3-10 demonstrates the anatomy of the authoring page in the basic mode.

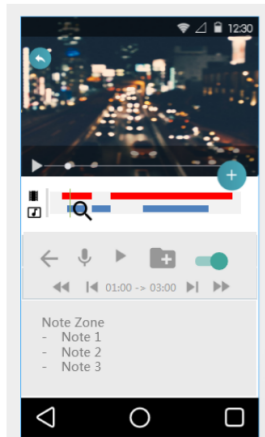


YD Authoring Basic mode page

- 1 Video window
- 2 Video & clips progress seek bar, with magnifier(red: video, blue: clips)
- 3 Existing clips lists (scrollable)
- 4 Notes zone (scrollable)

Figure 3-10 YouDescribe Authoring Page Basic Mode

Authoring page record mode is the mode where an author will enter when he taps the “add new” button. The functions it provides include “adding/editing textual notes to assist further recording” and “recording one new audio clips”. Figure 3-11 demonstrates the anatomy of the authoring page in the recording mode



YD Authoring Recording mode page

- 1 Video window
- 2 Video & clips progress seek bar, with magnifier (red: video, blue: clips)
- 3 Recording panel
- 4 Notes zone (scrollable)

Figure 3-11 YouDescribe Authoring Page Recording Mode

Authoring page editing mode is the mode where an author will enter when he taps the “editing” button (the pencil icon) on any item in the “existing clips list”. The functions it provides include “adding/editing textual notes to assist further recording” and “editing existing audio clip”. Figure 3-11 demonstrates the anatomy of the authoring page in the recording mode

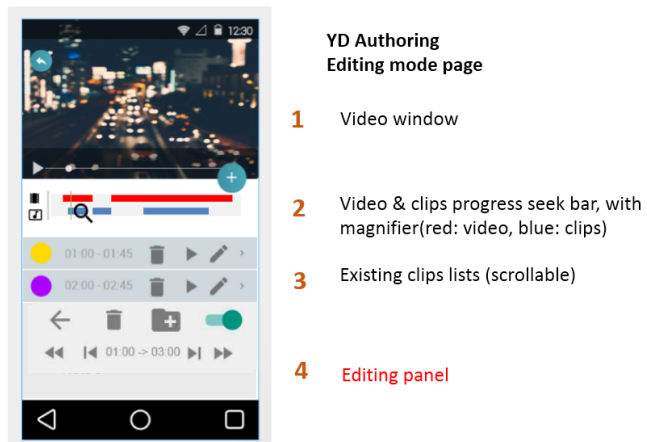


Figure 3-12 YouDescribe Authoring Page Editing Mode

Authors can switch between these three modes by tapping some specific buttons. Users can enter recording mode by tapping the “add new” button from the basic mode and return back to the basic mode by tapping the “return button” on the record panel. (Figure 3-13)

Likewise, users will enter the editing mode by tapping the editing button (pencil icon) on some specific item in the existing clip list. Returning to the basic mode is achievable by tapping the return button. (Figure 3-14)

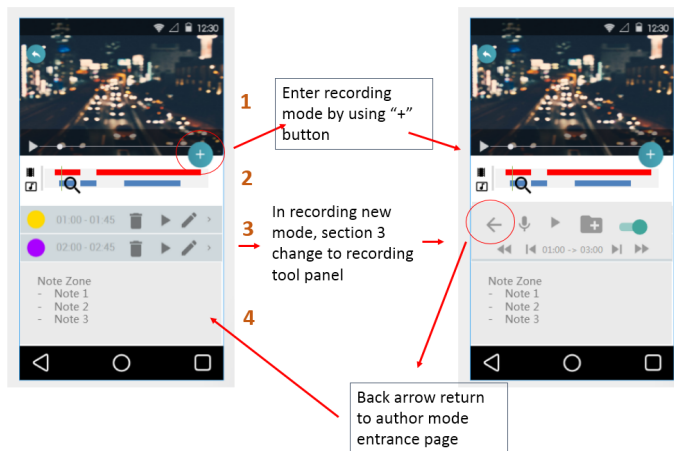


Figure 3-13 Switch between Basic Mode and Recording Mode

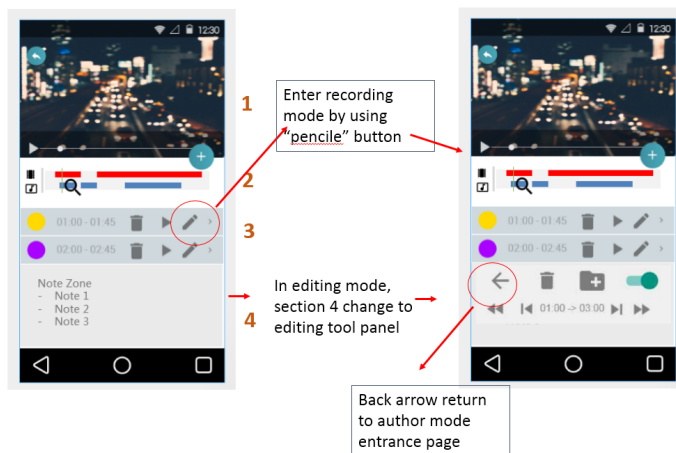


Figure 3-14 Switch between Basic Mode and Editing Mode

Figure 3-15 demonstrates the comparison of the authoring pages under the three modes. Under different scenarios, the authoring page is composed of a combination of sub-modules. Switching between different modes can be performed by replacing some sub-module with others.

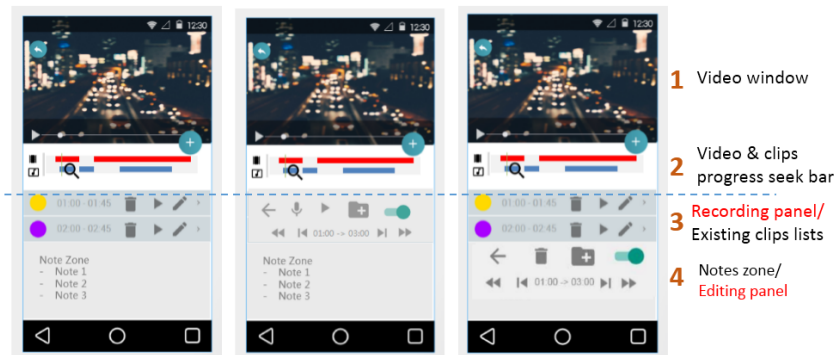


Figure 3-15 Anatomy of the Authoring Page

4 Implementation

The YouDescribe Android app is developed with Android Studio, in which the primary language we use is Java.

Android [19] is a mobile operating system developed by Google, based on the Linux kernel and designed primarily for touchscreen mobile devices such as smartphones and tablets. In addition, Google has further developed Android TV for televisions, Android Auto for cars, and Android Wear for wrist watches, each with a specialized user interface. Variants of Android are also used on game consoles, digital cameras, PCs and other electronics.

Initially developed by Android Inc., which Google bought in 2005, Android was unveiled in 2007, with the first commercial Android device launched in September 2008. The operating system has since gone through multiple major releases, with the current version being 8.1 "Oreo", released in December 2017. Android has been the best-selling OS on tablets since 2013. As of May 2017, it has two billion monthly active users and the largest installed base of any operating system. As of February 2017, Google Play store features over 2.7 million apps.

Java [18] is a general-purpose computer programming language that is concurrent, class-based, object-oriented, and specifically designed to have as few implementation dependencies as possible. It is intended to let application developers "write once, run anywhere" (WORA), meaning that compiled Java code can run on all platforms that support Java without the need for recompilation.

We developed YouDescribe Android app using Android Studio 3.2 as the primary IDE. **Android Studio** is the official integrated development environment (IDE) for Google's Android operating system, built on JetBrains' IntelliJ IDEA software and designed specifically for Android development. It is available for download on Windows, macOS and Linux based operating systems. It is a replacement for the Eclipse Android Development Tools (ADT) as primary IDE for native Android application development. Android Studio was announced on May 16, 2013 at the Google I/O conference. It was in early access preview stage starting from version 0.1 in May 2013. The current stable version is 3.2 released in Sep. 2018.

4.1 Application Architecture

As we discussed in Chapter 3, the YouDescribe Android app is composed of three major pages. Accordingly, we organized the codes of the implementation in a similar modular approach. As Figure 4-1 shows, each page of the app is implemented by a module which has multiple related activities, fragments and classes. We also wrapped the data of video, audio description and audio clips in separated classes within a module named “model”. Some static methods that are widely used through multiple activities or classes are wrapped in corresponding utility classes, which are all wrapped in the “utils” module. Given this, we constructed a structure with three major UI-related modules for each page, as well as two utility modules sharing all UI modules. Actual code structure within each module is shown in Figure 4-2.

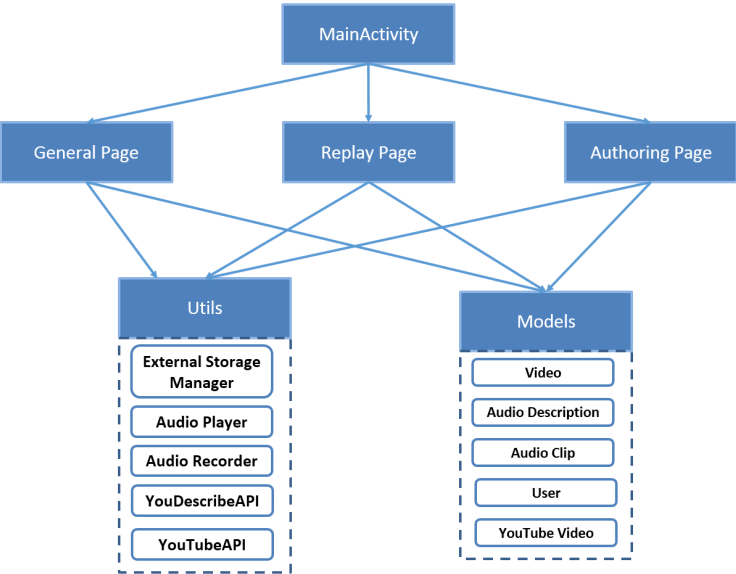


Figure 4-1 Code Structure of YouDescribe Android App

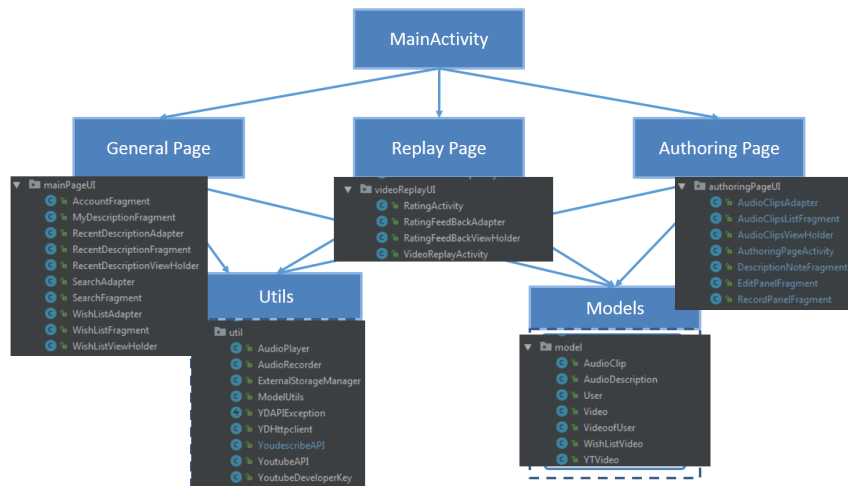


Figure 4-2 Codes within Modules

4.2 General Page

As demonstrated in Figure 3-3, the general page is composed of 5 subpages. We leverage a “Bottom Navigation View” along with 5 Fragments to implement this design.

“**Bottom Navigation View**” represents a standard bottom navigation bar for application. It is an implementation of material design bottom navigation. Bottom navigation bars make it easy for users to explore and switch between top-level views in a single tap. It is often used when application has three to five top-level destinations.

Each sub-view within the general page is implemented with a fragment. A **Fragment** represents a behavior or a portion of user interface in an Activity. Developers can combine multiple fragments in a single activity to build a multi-pane UI and reuse a fragment in multiple activities. A fragment can be considered as a modular section of an activity, which has its own lifecycle, receives its own input events, and can be added or removed while the activity is running (sort of like a "sub activity" that can be reused in different activities).

Switching in “Bottom Navigation View” is operated by tapping the corresponding icon on the bottom navigation bar. In addition, we added a gesture of swipe to switch between the neighboring sub-views. This feature is implemented with ViewPager. **ViewPager** is most often used in conjunction with Fragment, which is a convenient way to supply and manage the lifecycle of each page. There are standard adapters implemented for using fragments with the ViewPager, which cover the most common use cases.

The code implementing the switching operations is listed in the following figures.

```
this.mBottomNavigationView.setOnNavigationItemSelectedListener(new BottomNavigationView.OnNavigationItemSelectedListener() {
    @Override
    public boolean onNavigationItemSelected(@NonNull MenuItem item) {
        switch (item.getItemId()) {
            case R.id.recent_descriptions:
                mViewPager.setCurrentItem(0);
                break;

            case R.id.wishlist:
                mViewPager.setCurrentItem(1);
                break;

            case R.id.my_descriptions:
                mViewPager.setCurrentItem(2);
                break;

            case R.id.search_video:
                mViewPager.setCurrentItem(3);
                break;

            case R.id.account:
                mViewPager.setCurrentItem(4);
                break;
        }
        return true;
    }
});
```

Figure 4-3 Switching with Bottom Navigation View

```

public class SectionsPagerAdapter extends FragmentPagerAdapter {
    private static final int TAB_NUMS = 5;

    public SectionsPagerAdapter(FragmentManager fragmentManager) { super(fragmentManager); }

    @Override
    public Fragment getItem(int position) {
        switch (position) {
            case 0:
                return new RecentDescriptionFragment();
            case 1:
                return new WishListFragment();
            case 2:
                return new MyDescriptionFragment();
            case 3:
                return new SearchFragment();
            case 4:
                return new AccountFragment();
            default:
                return null;
        }
    }
}

```

Figure 4-4 Switching with ViewPager

4.2.1 Recent Description Sub-Page

Recent Description page (Figure 3-4) is the one which lists all the videos that have audio descriptions for replay. The list is ordered by the date of the audio descriptions when they are added. Recent Description Page applied an infinite loading list as its foundation. The structure of this page is demonstrated in the following figure.

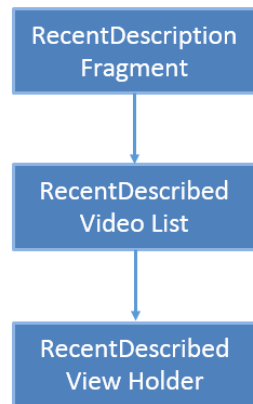


Figure 4-5 Structure of Recent Described Page

Infinite loading list is a basic UI component that is widely used throughout YouDescribe Android app. The idea of infinite loading is to use a special view holder in the list as the indicator to launch a new round of data fetching. The special view holder is always attached behind all other normal view holders. Whenever it is displayed, a special YouDescribe API call will be launched and newly retrieved data will be attached behind old normal data. Then the special view holder will be added behind these new data.

Code snippet in Figure 4-6 demonstrates this approach. The core implementation is wrapped in the `InfiniteListAdapter`. The recent described list creates a concrete instance of that adapter with a `loadmorelistener` instance. Then the recent described list is attached to that adapter instance of the `recyclerview` to turn a normal `recyclerview` into the infinite loading recycler view, as Figure 4-7 shows.

```

@Override
public void onBindViewHolder(BaseViewHolder holder, int position) {
    final int viewType = getItemViewType(position);
    if (viewType == TYPE_LOADING) {
        loadMoreListener.onLoadMore();
    } else {
        onBindItemViewHolder(holder, position);
    }
}
}

```

Figure 4-6 Code Snippet for Infinite Loading List

```

adapter = RecentDescriptionAdapter.getRecentDescriptionAdapter(onLoadMore, recentDescriptionFragment.this);
recyclerView.setAdapter(adapter);
}

```

Figure 4-7 Linked the Infinite Loading Adapter to Recycler View

The callback function to enter the replay page for each view holder is defined in the method that binds the view holder in the adapter class.

```

private void bindRecentDescribedView(RecentDescriptionViewHolder viewHolder, final Video describedVideo) {
    viewHolder.recentVideoListTitle.setText(describedVideo.getTitle());
    viewHolder.recentVideoListSubtitle.setText(describedVideo.getAuthor());
    Picasso.get().load(Paths.get("http://img.youtube.com/vi/" + describedVideo.getYoutube_id() + "/0.jpg"))
        .placeholder(R.mipmap.ic_launcher_yd)
        .error(R.mipmap.ic_launcher_yd)
        .into(viewHolder.recentVideoListThumbnail);

    /*Tap on the entire row*/
    viewHolder.cover.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            Intent intent = new Intent(mRecentDescriptionFragment.getContext(), VideoReplayActivity.class);
            /*Choose 0 ad as default*/
            AudioDescription ad = describedVideo.getAudio_descriptions().get(0);
            intent.putExtra(VideoReplayActivity.DESCRIBED_VIDEO,
                ModelUtils.toString(describedVideo, new TypeToken<Video>() {
                });
            intent.putExtra(VideoReplayActivity.AUDIO_DESCRIPTION,
                ModelUtils.toString(ad, new TypeToken<AudioDescription>() {
                });
            getContext().startActivity(intent);
        }
    });
}
}

```

Figure 4-8 Callback Function Binding for Each View Holder

4.2.2 Wish list sub-page and my description sub-page

Wish list sub-page (Figure 3-5) and my descriptions sub-page (Figure 3-6) are both the lists that are oriented to the authors. Any signed-in user who wants to contribute a new audio description can pick a video of interest from the “Wish List sub-page” and tap the “Describe” button for that video to contribute a new audio description. “My description” list contains the video that the current signed-in user has described. Starting a new description from the “Wish list” will add the video to the “my description” list of the user. To simplify the logic of user – description pair, we assume that one user can only contribute one description to a specific video, meaning that each item on the “my descriptions” is unique.

Like “recent description” sub-page, wish list and my descriptions are mostly constructed by infinite loading list. The only difference is that the viewholder for each kind of list We change the view holders accordingly, and set the callback function of the “Describe” button as the trigger of the authoring page.

Deleted: between wish list and my description

Deleted: the view holder is not like it for the recent description.

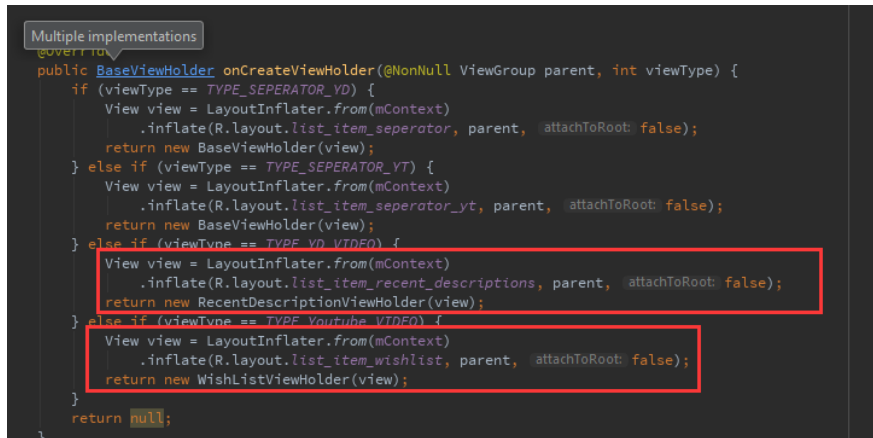
```
/*Bind event for each button*/
wishListViewHolder.wishListDescribe.setOnClickListener((v) -> {
    if (ModelUtils.checkPreference(getContext(), ModelUtils.GLOBAL_USER_INFO)) {
        Intent intent = new Intent(getContext(), AuthoringPageActivity.class);
        intent.putExtra(AuthoringPageActivity.YOUTUBE_ID, wishListVideo.getYoutube_id());
        intent.putExtra(AuthoringPageActivity.YOUTUBE_TITLE, wishListVideo.getYoutube_title());
        mWishListFragment.startActivity(intent);
    } else {
        Toast.makeText(mWishListFragment.getActivity(), text: "Please sign in to make description.", Toast.L
    }
}
```

Figure 4-9 Callback Function Binding for “Describe” Button to Trigger Authoring Page

4.2.3 Search sub-page

Search sub-page (Figure 3-7) is to list videos (with descriptions and without descriptions respectively). Entering any keyword will trigger a query to YouDescribe API and YouTube API simultaneously. YouDescribe API will return the videos that have descriptions related to that keyword. People can directly replay those videos from the search result (enter the replay page). YouTube API will return the relevant videos that have not been described. Users can describe those videos by tapping the “describe” tag, like what they can do in the wish list page.

This page is composed of two recycle view lists with different view holders. The view holders from “recent description” page and “wish list” page are reused here.

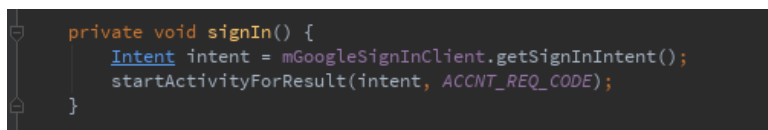


```
Multiple implementations
@Override
public BaseViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
    if (viewType == TYPE_SEPARATOR_VD) {
        View view = LayoutInflater.from(mContext)
            .inflate(R.layout.list_item_seperator, parent, attachToRoot: false);
        return new BaseViewHolder(view);
    } else if (viewType == TYPE_SEPARATOR_YT) {
        View view = LayoutInflater.from(mContext)
            .inflate(R.layout.list_item_seperator_yt, parent, attachToRoot: false);
        return new BaseViewHolder(view);
    } else if (viewType == TYPE_VD_VIDE0) {
        View view = LayoutInflater.from(mContext)
            .inflate(R.layout.list_item_recent_descriptions, parent, attachToRoot: false);
        return new RecentDescriptionViewHolder(view);
    } else if (viewType == TYPE_Youtube_VIDE0) {
        View view = LayoutInflater.from(mContext)
            .inflate(R.layout.list_item_wishlist, parent, attachToRoot: false);
        return new WishListViewHolder(view);
    }
    return null;
}
```

Figure 4-10 Recent Description View Holder and Wish List View Holder in Search Page

4.2.4 Account page

Account page (Figure 3-8) is prepared for users to log in and log out. We leverage the Google Sign-In library to facilitate the development. Tapping the log in button will simply start a new Google Sign-In activity and all following operations are handled by the Google Sign-In activity.



```
private void signIn() {
    Intent intent = mGoogleSignInClient.getSignInIntent();
    startActivityForResult(intent, ACCNT_REQ_CODE);
}
```

Figure 4-11 Use Google Sign-In Library to Sign In

4.3 Replay Page

Tapping an item on the “Recent Describe” list will take the user to the “Replay page”. This section introduces the implementation of the replay page and some key algorithms and strategies used for some important features.

4.3.1 Replay of YouTube video

Replay page leverages the official YouTube Player Android API (<https://developers.google.com/youtube/android/player/reference/com/google/android/youtube/player/package-summary>) as the player embedded in the replay page. In particular, we embedded a `YouTubePlayerFragment` within the Replay page (`VideoReplayActivity`) as the replay window of the YouTube video (Figure 3-9). In the meantime, we enclosed a `YouTubePlayer` instance within the `VideoReplayActivity` as the core replay controller of the video. All video-related operations, such as pause, start, and stop are methods of this player. Meanwhile, the replay also provides status observation method which we can leverage to monitor the status of the video replay process. The initialization process of the `YouTubePlayerFragment` and `YouTubePlayer` consists of two steps. We first create an initialization listener to register the operations that need to be executed after initialization. We can store the YouTube player as the member field in this method, and also start the caching of the video that will be played here. Then we launch the initialization method using this listener as well as the YouTube API key that we register before. The code snippet and the workflow are demonstrated in the following figure.

Initialize the YouTube fragment

```
public void loadYoutube() {  
    YouTubePlayerFragment youtubePlayerFragment =  
        (YouTubePlayerFragment) getFragmentManager().findFragmentById(R.id.youtube_replay_fragment);  
    youtubePlayerFragment.initialize(YouTubeAPI.DEVELOPER_KEY, onInitializedListener: this);  
}
```

- Retrieve the YouTube player as the core replay controller in the callback of initialization
- Cue the video from YouTube by YouTube id

```
@Override  
public void onInitializationSuccess(YouTubePlayer.Provider provider, YouTubePlayer youtubePlayer, boolean wasRestored) {  
    this.mYouTubePlayer = youtubePlayer;  
    this.mYouTubePlayer.setPlayerStateChangeListener(this);  
    this.mYouTubePlayer.setPlaybackEventListener(this);  
    this.mYouTubePlayer.setPlayerStyle(YouTubePlayer.PlayerStyle.CHROMELESS);  
    if (!wasRestored) {  
        youtubePlayer.cueVideo(this.mVideo.getYoutube_id());  
    }  
}
```

Figure 4-12 Code Snippet for YouTubeFragment Initialization

4.3.2 Coordination of audio clips and YouTube video

The main purpose of the replay page is to coordinate the replay of YouTube video and the replay of the audio clips properly, so that one audio clip can precisely describe the motions of the YouTube video on the screen. The key data for this coordination is the time that each audio clip starts during the replay of the YouTube video. It is the “starting time” data field of the “AudioClip” data model. We leveraged an infinite looping thread aside from the main UI thread monitoring the current time (in seconds) from the start of the video replay. As long as the current time is the starting time for a specific “AudioClip”, we launch the replay of that “AudioClip” on the main thread. This process is demonstrated in Figure 4-13.

For the static wav files and the metadata of the audio clip, they are all downloaded ever since users enter the replay page. We leveraged a new library called “Fetch” (<https://github.com/tonyofrancis/Fetch>) to download the static files for its high efficiency. The file paths and metadata are stored in a map (hashtable) in which we use the starting time of every file as the key for the quick lookup.

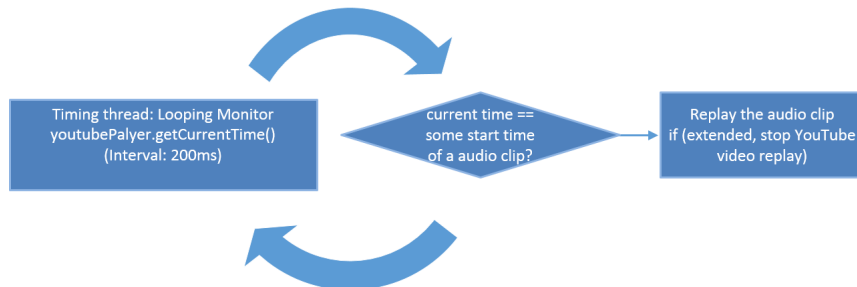


Figure 4-13 Strategy to coordinate the replay of audio clips

YouTubeplayer provides a method “getCurrentTime()” to get the playback duration from the start. This is what we need in the monitoring thread. Figure 4-14 shows the code snippet that monitors the playback progress and launches the corresponding audio clips.

```

/*Monitoring the play status */
public void run() {
    while (mYouTubePlayer.isPlaying()) {
        try {
            Thread.sleep( millis: 200);
            if (!mYouTubePlayer.isPlaying()) break; // special detect for pause button!
            /*Detect if current player met a point need to replay the AudioClips*/
            final double currentTime = (double) (mYouTubePlayer.getCurrentTimeMillis() / 1000);
            Log.d( tag: TAG + " Monitor", msg: "Detecting..." + currentTime);
            if (acTimeTable.containsKey(currentTime) && !finishedTrack.containsKey(currentTime)) {
                Log.d( tag: TAG + " Player", msg: "Play" + acTimeTable.get(currentTime)[0]);
                handler.post(new Runnable() {
                    @Override
                    public void run() {
                        /*Gain audio focus
                        * https://developer.android.com/guide/topics/media-apps/audio-focus#audio_focus_pre_android_80 */
                        mAudioManager.requestAudioFocus( null, AudioManager.STREAM_MUSIC, AudioManager.AUDIOFOCUS_GAIN_TRANSIENT_MAY_DUCK);
                        playAudioClips(acTimeTable.get(currentTime)[0]); // play video
                        VideoReplayActivity.this.audioShouldPlay = true;
                        if (acTimeTable.get(currentTime)[1].equals("extended")) {
                            mYouTubePlayer.pause();
                            VideoReplayActivity.this.videoShouldPlay = false;
                        }
                    }
                });
                finishedTrack.add(currentTime);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            Log.d( tag: TAG + " Monitor", msg: "Monitor interrupted in sleep");
        }
    }
}

```

Figure 4-14 Code snippet for the Monitoring thread

We implemented two important interfaces: “YouTubePlayer.PlaybackEventListener”

(<https://developers.google.com/youtube/android/player/reference/com/google/android/youtube/player/YouTubePlayer.PlaybackEventListener>) [24 - 30] and “YouTubePlayer.PlayerStateChangeListener”

(<https://developers.google.com/youtube/android/player/reference/com/google/android/youtube/player/YouTubePlayer.PlayerStateChangeListener>)

[TubePlayer.PlayerStateChangeListener](#)) [30 - 37] as the callback hooks to respond to related events in the playback page. Figure 4-15 demonstrates the callback functions to trigger the monitoring loop thread.

“onVideoStarted” is called as soon as the video starts. However, in the cases when there is advertisement before the real video, “onVideoStarted” cannot tell if the currently ongoing video is the real one or the advertisement before the real video. Therefore, we leveraged the “onPlaying” method of the YouTube player in addition to “onVideoStarted” to detect if the actual video is really in play.

Callback function to trigger the monitor thread

```
@Override
public void onVideoStarted() {
    /*Only triggered at the first time playback started
    * Used to skip Ads*/
    this.videoStarted = true;
}

/*Youtube Player PlayStateChange & PlaybackEvent*/
@Override
public void onPlaying() {
    if (videoStarted) {
        VideoReplayActivity.this.videoShouldPlay = true; // indicate that current video should be playing
        new Thread( target: this).start();
    }
}
```

Figure 4-15 Callback functions to trigger the looping monitoring thread

4.4 Authoring page

As described in the design section of the authoring page (section 3.2.3), the authoring page has to work in three different modes (basic mode, recording mode and editing mode). In fact, the authoring page is basically composed of 5 different components. Each working mode is a combination of some components out of these five, which is shown in Figure 4-16 and the following table.

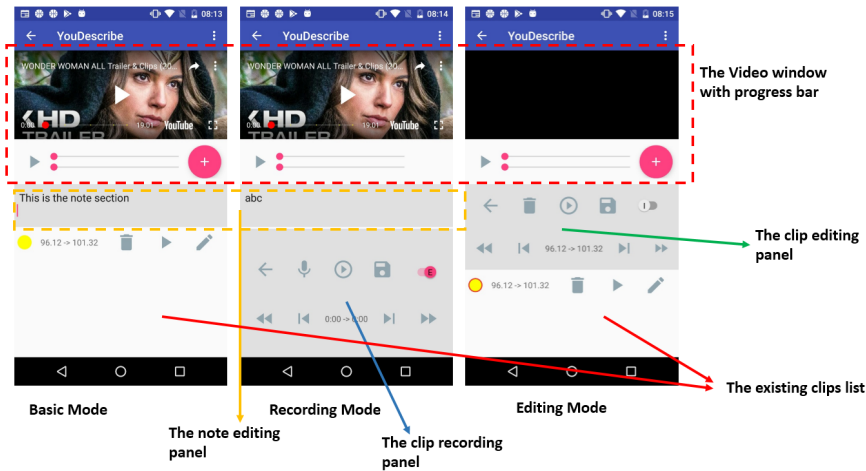


Figure 4-16 Authoring page under different modes

Table 4-1 Anatomy of the three working models

Working Mode	Components
Basic mode	The Video window with progress bar The note editing panel The existing clips list
Recording mode	The Video window with progress bar The note editing panel The clip recording panel
Editing mode	The Video window with progress bar The clip editing panel The existing clips list

We implemented the authoring page as the “AuthoringPageAcitivity” which hosts the five different components. Under different scenario, the authoring page shall display as a combination of different UI

components. From the above table, it is clear that “The video window with progress bar” is persistent throughout all the modes. Therefore, we embedded that component as a static one, which we did not give a separate class. For the other four components, we implement each of them as a separate “Fragment”.

Table 4-2 Fragment for each authoring page component

Component	Fragment
The note editing panel	DescriptionNodeFragment
The existing clips list	AudioClipsListFragment
The clip recording panel	RecordPanelFragment
The clip editing panel	EditPanelFragment

Activity class provides a series of fragment management APIs to control the occurrence of the combination of some specific fragments.

We created a singleton instance for each fragment of the four components as soon as the AuthoringPageActivity is launched.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    this.youtubeID = getIntent().getStringExtra(YOUTUBE_ID);
    this.youtubeTitle = getIntent().getStringExtra(YOUTUBE_TITLE);
    this.currentUser = ModelUtils.read(context: this, ModelUtils.GLOBAL_USER_INFO, ModelUtils.GLOBAL_TYPE);
    setContentView(R.layout.activity_authoring);
    ButterKnife.bind(target: this);
    setSupportActionBar(mToolbar);
    getSupportActionBar().setDisplayHomeAsUpEnabled(true);
    getSupportActionBar().setDisplayHomeAsUpEnabled(true);

    /*Create four fragments*/
    this.mRecordPanelFragment = RecordPanelFragment.getRecordPanelFragment(this);
    this.mDescriptionNoteFragment = DescriptionNodeFragment.getDescriptionNoteFragment(this);
    this.mAudioClipsListFragment = AudioClipsListFragment.getAudioClipsListFragment(this);
    this.mEditPanelFragment = EditPanelFragment.getEditPanelFragment(this);

    /*Init fragment container*/
}
```

Figure 4-17 Create singleton instance for each fragment

Then we initialized the authoring page with the basic mode to attach the note panel and the exiting list to the authoring page.

```
/*Init fragment container*/
FragmentTransaction fragmentTransaction = this.getSupportFragmentManager().beginTransaction();
fragmentTransaction.add(R.id.fragment_1, mDescriptionNoteFragment);
fragmentTransaction.add(R.id.fragment_2, mAudioClipsListFragment);
fragmentTransaction.commit();
```

Figure 4-18 Initial fragments in authoring page

For any time when we need to switch to a different mode, we switch some fragment with the target fragment. The following snippet (Figure 4-19) shows the operation when we switch from the basic mode into the recording mode (tapping the “creating new clips” button). This snippet basically conducts a fragment replacement, in which we replace the instance of “AudioClipsListFragment” with the instance of “RecordPanelFragment”. This operation is registered as the click listener of the “addnewclips” button.

In terms of data model, we have an “AudioDescription” instance hosted in the AuthoringPageActivity. This “AudioDescription” instance is accessible throughout all the fragments via accessing the “AuthoringPageActivity” instance. We will describe the details of the implementation of each fragment in the following part of this chapter.

```
this.mEditPanelFragment = EditPanelFragment.getEditPanelFragment(this);

/*Init fragment container*/
FragmentTransaction fragmentTransaction = this.getSupportFragmentManager().beginTransaction();
fragmentTransaction.add(R.id.fragment_1, mDescriptionNoteFragment);
fragmentTransaction.add(R.id.fragment_2, mAudioClipsListFragment);
fragmentTransaction.commit();

addNewClipButton.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        FragmentTransaction fragmentTransaction = getSupportFragmentManager().beginTransaction();
        fragmentTransaction.setCustomAnimations(R.anim.slide_in_right, R.anim.slide_out_left);
        fragmentTransaction.replace(R.id.fragment_2, mRecordPanelFragment);
        fragmentTransaction.commit();
        addNewClipButton.setVisibility(View.INVISIBLE);
        Log.d(TAG, ExternalStorageManager.getExternalPrivateStorageFolderPath(AuthoringPageActivity.this, youtube));
    }
});
```

Figure 4-19 Fragment replacement for mode switching

4.4.1 The note editing panel

The note editing panel is implemented by “DescriptionNodeFragment”. This fragment is an edit text view that allows users to input the textual notes. We assume that the text persists through the entire recording or editing process. We implemented the “afterTextChanged” method in the “TextWatcher” interface which registered as the text change listener of the edit text view. Once the text is changed, we persist the textual content to the “note” field of the “AudioDescription” instance held in “AuthoringPageActivity” instance, as shown in Figure 4-20.

```
@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
    text.addTextChangedListener(new TextWatcher() {
        @Override
        public void beforeTextChanged(CharSequence charSequence, int i, int i1, int i2) {
        }

        @Override
        public void onTextChanged(CharSequence charSequence, int i, int i1, int i2) {
        }

        @Override
        public void afterTextChanged(Editable editable) {
            if (editable != null && editable.toString().length() > 0) {
                mAuthorPageActivity.currentAudioDescription.setNotes(editable.toString());
            }
        }
    });
}
```

Figure 4-20 Snippet for persisting notes

4.4.2 The existing clip list

The existing clip list (AudioClipsListFragment) holds all the audio clips under the current “Audio Description”. Given that it is essentially a list, we leveraged the recycler view as the core layout of this fragment. Tapping the editing button (pencil icon) on the item view holder will launch a fragment switch to replace the note editing panel with the clip editing panel (Figure 4-21).

```
audioClipsViewHolder.clipEditButton.setOnClickListener((v) -> {  
    mAuthorPageActivity.currentAudioClipId = position; // set currentEditingAudioClips id  
    audioClipsViewHolder.clipTypeView.setImageResource(currAudioClip.playback_type.equals("extended") ? R.drawable.purple : R.drawable.pink);  
    if (mContext instanceof AppCompatActivity) {  
        LinearLayout.LayoutParams params = (LinearLayout.LayoutParams) ((AppCompatActivity) mContext).findViewById(R.id.fragment_1).params;  
        params.weight = 3.0f;  
        ((AppCompatActivity) mContext).findViewById(R.id.fragment_1).setLayoutParams(params);  
  
        FragmentTransaction fragmentTransaction = ((AppCompatActivity) mContext).getSupportFragmentManager().beginTransaction();  
        fragmentTransaction.setCustomAnimations(R.anim.slide_in_right, R.anim.slide_out_left);  
        fragmentTransaction.replace(R.id.fragment_1, ((AuthoringPageActivity) mContext).mEditPanelFragment);  
        fragmentTransaction.commit();  
    }  
});
```

Figure 4-21 Editing button listener: switch the note panel with the clip editing panel

4.4.3 The clip recording panel

The clip recording panel (RecordPanelFragment) holds all the tools that are necessary to record a new audio clip (Figure 4-16, the clip recording panel). It is launched by tapping the big plus button at the right bottom of the video replay fragment. The major functions provided in this panel are start recording, stop recording, and saving the new recorded audio clip. More details on the logic and necessary operation are described in the following subsections.

A) Start recording

Recording is started by tapping the recording button (the microphone icon). Upon the tapping, the state of the record panel will change from “Prepare to record” into “In Recording”. Meanwhile, the current YouTube video playback timestamp will be recorded as the starting time of the new audio clip. If the new audio clip is an extended one, we pause the YouTube player if it is still in replay. On the other hand, if the new audio clips are in inline type, we resume the replay if the YouTube player is not playing. We change

the color of the microphone icon into red to indicate that YouDescribe is in recoding state. We prepare the file path for the new recorded clip in the local file system on the phone, and record the audio input through the microphone of the phone using the utility class “AudioRecorder”. Meanwhile, we start a separated time thread out of the current UI thread to record the length of the new recorded path.

```
isRecording = false;
if (recordPanelState == PREPARE_RECORD) {
    recordPanelState = IN_RECORDING;
    /*get Start time & control the play of ori youtube video*/
    this.startTime = (double) (this.mAuthoringPageActivity.mAuthoringPageYouTubePlayer.getCurrentTimeMillis()) / 1000;
    this.endTime = this.startTime;
    if (this.isExtended) {
        if (this.mAuthoringPageActivity.mAuthoringPageYouTubePlayer.isPlaying())
            this.mAuthoringPageActivity.mAuthoringPageYouTubePlayer.pause();
    } else {
        if (!this.mAuthoringPageActivity.mAuthoringPageYouTubePlayer.isPlaying())
            this.mAuthoringPageActivity.mAuthoringPageYouTubePlayer.play();
    }

    RecordPanelFragment.this.recordRecordButton.setImageResource(R.drawable.ic_mic_red_500_36dp);
    isRecording = true;
    currentClipPath = ExternalStorageManager.createRecordFilePath(getContext(), currentVideo.getYoutube_id());
    AudioRecorder.startRecord(currentClipPath);

    /*Start the timing thread*/
    mCountTimeThread = new Thread( target: this);
    mCountTimeThread.start();
    /**/
}
```

Figure 4-22 Code snippet when we start recording

```
@Override
public void run() {
    while (recordPanelState == IN_RECORDING) {
        try {
            Thread.sleep( millis: 100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        this.endTime += 0.1;
    }
}
```

Figure 4-23 Code snippet for the timing thread to track the duration of the recording

B) Stop recording

Recording can be stopped by tapping the microphone icon again. This operation will change the state of the record panel from “In Recording” to “Recording finished”. The audio recorder will be stopped and

released. The timing thread will be terminated, with the ending time of the recorded clip being saved. Some important data fields are saved directly into the data model of the current “AudioClip” instance (type, starting and ending time as well as the duration of the clip). Since recorded file is now in the format of m4a, we still need a converter to change it into the wav format. After all these steps are finished, we now have a local audio clip residing in the file system of the phone, waiting for the user to upload to the backend server.

```
recordPanelState = RECORD_FINISHED;
isRecording = false;
RecordPanelFragment.this.recordRecordButton.setImageResource(R.drawable.ic_mic_blue_grey_300_36dp);
AudioRecorder.stopAndRelease();
if (this.mAuthoringPageActivity.mAuthoringPageYouTubePlayer.isPlaying()) {
    this.mAuthoringPageActivity.mAuthoringPageYouTubePlayer.pause();
}

currentAudioClip = new AudioClip(); // create audioClip
currentAudioClip.playback_type = this.isExtended ? "extended" : "inline"; //
currentAudioClip.audio_description = this.mAuthoringPageActivity.currentAudioDescription.getId(); //the current AD
currentAudioClip.start_time = String.valueOf(this.startTime);
currentAudioClip.end_time = String.valueOf(this.endTime);
currentAudioClip.duration = String.valueOf(this.endTime - this.startTime);

/*Update the start - end text field*/
String start = String.format("%.2f", startTime);
String end = String.format("%.2f", endTime);
if (this.isExtended) {
    this.recordClipStartEnd.setText(start + " -> " + start);
} else {
    this.recordClipStartEnd.setText(start + " -> " + end);
}
this.startTime = 0;
this.endTime = 0;

if (new File(currentClipPath).exists()) {
    RecordPanelFragment.this.recordReplayButton.setImageResource(R.drawable.ic_play_circle_outline_red_500_36dp);
    RecordPanelFragment.this.recordSaveButton.setImageResource(R.drawable.ic_save_red_500_36dp);
}

//convert m4a to wav for compatibility with YouTube
File file = new File(currentClipPath);
AndroidAudioConverter.with(this.getContext())
    .setFile(file)
    .setFormat(AudioFormat.WAV)
    .setCallback(mIConvertCallback)
    .convert();
```

Figure 4-24 Code snippet when we stop recording

C) Save (upload) recording

Saving task is to upload the local static audio file and the audio clip metadata to the backend server to persist them. This is achieved by starting an asynchronized task with a YouDescribe API with POST method. The payload of this POST YouDescribe API call includes the metadata (an instance of “AudioClip”) and the static file (Figure 4-26).

```

public void switchStateSaveButton() {
    if (recordPanelState == RECORD_FINISHED) {
        recordPanelState = PREPARE_RECORD;

        /*Upload task here*/
        new UploadFileTask(currentAudioClip, currentVideo, currentUser, currentAudioDescription).execute();
        AsyncTaskCompat.executeParallel(new UploadFileTask(currentAudioClip, currentVideo, currentUser, currentAudioDescription));
        /*Change button*/
        RecordPanelFragment.this.recordReplayButton.setImageResource(R.drawable.ic_play_circle_outline_blue_grey_300_36dp);
        RecordPanelFragment.this.recordSaveButton.setImageResource(R.drawable.ic_save_blue_grey_300_36dp);
        RecordPanelFragment.this.recordClipStartEnd.setText("00:00 -> 0:00");
    }
}

```

Figure 4-25 Start an UploadFileTask for uploading the local static file and metadata

```

public static String postNewAudioClips(AudioClip audioClip, Video video, AudioDescription audioDescription, User currentUser) throws YDAPIException {
    RequestBody requestBody = new MultipartBody.Builder()
        .setType(MultipartBody.FORM)
        .addFormDataPart("userId", currentUser.getId())
        .addFormDataPart("userToken", currentUser.getToken())
        .addFormDataPart("audioDescriptionId", audioClip.audio_description)
        .addFormDataPart("audioDescriptionNotes", audioDescription.getNotes())
        .addFormDataPart("title", video.getTitle())
        .addFormDataPart("description", video.getDescription())
        .addFormDataPart("playbackType", audioClip.playback_type)
        .addFormDataPart("label", "1") //
        .addFormDataPart("startTime", audioClip.start_time) //
        .addFormDataPart("endTime", audioClip.end_time) //
        .addFormDataPart("duration", audioClip.duration) //
        .addFormDataPart("wavfile", audioClip.file_path,
            RequestBody.create(MEDIA_TYPE_WAV, new File(audioClip.file_path)))
        .build();

    Request request = new Request.Builder()
        .url(AUDIOCLIPS_END_POINT + video.getyoutube_id())
        .post(requestBody)
        .build();

    try {
        Response response = YDHttpClient.getHttpClient().newCall(request).execute();
        String res = response.body().string();
        Log.d(TAG, "img: 'Upload succeed'");
        Log.d(TAG, res);
        return res;
    }
}

```

Figure 4-26 Code snippet for asynchronous uploading task

4.4.4 The clip editing panel

The clip editing panel is almost the same as the clip recording panel except for the delete button whereas it is the recording button on the clip recording panel. We focused on the recording related functions in the previous section. Now in this section, we will introduce the functions that are related to editing the existing clips: nudging the starting time, change the type of the audio clip, and delete it. For those buttons / icons that are shared by both the editing panel and recording panel, their functions are the same. Introductions missed in the previous section can be found here.

A) Nudging

At the bottom of the editing panel as well as the recording panel, there is a series of buttons and a text field that are related to the starting time of the audio clip. The text field indicates the starting and ending time of the audio clips. For inline type, it is the exact starting and ending time during the replay of the original YouTube video since the inline type is played along with the YouTube video. On the other hand, the text filed for the extended type is just the starting time, since the YouTube video is paused during the replay of the audio clip.

There are two buttons on each side of the text field. It is easy to tell from the appearance that they are used to adjust the starting time forwardly and backwardly. Button with one arrow will increase / decrease the starting time by 0.5s, while buttons with two arrow will nudge by 1s.

```
public void timeAdjust(View view) {
    AudioClip currentAudioClip = this.mAuthoringPageActivity.currentAudioDescription.getAudio_clips().get(this.mAuthoringPageAc
    Double start = Double.parseDouble(currentAudioClip.start_time);

    if (view.getId() == R.id.edit_nudge_back_1s) {
        start -= 1;
    } else if (view.getId() == R.id.edit_nudge_back_05s) {
        start -= 0.5;
    } else if (view.getId() == R.id.edit_nudge_forward_1s) {
        start += 1;
    } else if (view.getId() == R.id.edit_nudge_forward_05s) {
        start += 0.5;
    }
    currentAudioClip.start_time = String.valueOf(start);
    currentAudioClip.end_time = String.valueOf(start + Double.parseDouble(currentAudioClip.duration));
    String startString = String.format("%.2f", start);
    String endString = String.format("%.2f", Double.parseDouble(currentAudioClip.end_time));
    if (currentAudioClip.playback_type.equals("extended")) {
        this.clipStartEnd.setText(startString + " -> " + startString);
    } else {
        this.clipStartEnd.setText(startString + " -> " + endString);
    }
}
```

Figure 4-27 Implementation of the nudging function

B) Type changing

As we've already known, there are two types of audio clips in YouDescribe: extended and inline. The toggle button on most right position of the panel is to adjust the type of the current audio clip. Simply toggle that button will change the type.

```
with button/  
s.editSwitchToggle.setOnCheckedChangeListener((compoundButton, isChecked) => {  
    if (compoundButton.getId() == R.id.edit_switch) {  
        mAuthorizingPageActivity.currentAudioDescription.getAudioClips().get(mAuthorizingPageActivity.currentAudioClipId).playback_type = isChecked ? "extended" : "inline";  
        Log.d("Record", mAuthorizingPageActivity.currentAudioDescription.getAudioClips().get(mAuthorizingPageActivity.currentAudioClipId).playback_type);  
    }  
})
```

Figure 4-28 Implementation of the type switch button

C) Deleting

When the author wants to totally dump the current audio clip, s/he can simply tap the deleting button to launch a DELETE request to the YouDescribe API to totally delete it locally and remotely. Upon a successful deleting operation, the authoring page will automatically return from the editing mode back to the basic mode, for the current audio clip does not exist any longer, and there is no need to stay in the editing mode of that audio clip.

```
public static void deleteAC(String acID, String userID, String userToken) throws YDAPIException {  
    JSONObject jsonObject = new JSONObject();  
    jsonObject.addProperty("userId", userID);  
    jsonObject.addProperty("userToken", userToken);  
    RequestBody body = RequestBody.create(MediaType.parse("application/json"), jsonObject.toString());  
    Response response = makeDeleteRequest(URI.AUDIOCLIPS_END_POINT + acID, body);  
}
```

Figure 4-29 Implementation of deleting task

4.5 Accessibility features

As an app whose target users are the blind or visually impaired people, YouDescribe must consider accessibility for any features that are designed for those people. The most prevalent tool to assist visually impaired in using the smart phone is screen reader. TalkBack is the Google screen reader included on Android devices. TalkBack gives you spoken feedback so that you can use your device without looking at

the screen. To turn on TalkBack, go to the Setting app of the device, open Accessibility and then go to TalkBack. Choose open TalkBack. Then users can explore the screen by touch, or swipe linearly to hear items in order. To explore by touch, slowly drag one finger around the screen. TalkBack announces the icons, buttons, and other items as you drag your finger over them. When the focus reaches an item you'd like to select, double-tap anywhere on the screen to select the focused item. We gave some basic gestures that users can refer to in the following table. For more advanced operations, people can refer to the official guidance of TalkBack (<https://support.google.com/accessibility/android/answer/6151827>, <https://support.google.com/accessibility/android/answer/6006598>).

Table 4-3 TalkBack Gestures

Action	Swipe
Move to next item on screen	Swipe right
Move to previous item on screen	Swipe left
Cycle through navigation settings	Swipe up or down
Select focused item	Double-tap
Move to first item on screen	Up then down
Move to last item on screen	Down then up
Scroll forward (if you're on a page longer than one screen)	Right then left
Scroll back (if you're on a page longer than one screen)	Left then right
Move slider up (such as volume)	Right then left
Move slider down (such as volume)	Left then right

There is a special textual field where we can specify what content the TalkBack will announce. It is called “android: contentDescription”, as Figure 4-30 shows. YouDescribe Android app has specified this textual field for every UI component. Users can understand the function of that button by listening to the talk back of that content description with TalkBack open on the phone.

```
android:layout_gravity="center"
android:orientation="horizontal">

<android.support.design.widget.FloatingActionButton
    android:id="@+id/youtube_replay_button"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_gravity="center"
    android:paddingTop="10dp"
    android:paddingBottom="10dp"
    android:paddingLeft="30dp"
    android:paddingRight="30dp"
    android:src="@drawable/ic_play_arrow_white_48dp"
    app:useCompatPadding="true"
    android:contentDescription="play video and description"
    app:layout_anchorGravity="end"/>
```

Figure 4-30 Attribute of contentDescription for TalkBack

5 Testing

This section shows the details related to the tests that have been conducted for the YouDescribe Android app.

5.1 Function Testing

Functional testing is a series testing that is to confirm that all the functional requirements (section 2.4) have been fulfilled.

Table 5-1 Function test cases

Case 1	Test Name	Search by Video Name
	Description	Search for videos by using the keyword in the search page
	Input	Navigate to the search tab and enter “yellow” in the search bar
	Expected output	Three described videos shows on under the Described Videos section, touching any video shall enter the replay page of that video; Meanwhile 30 non-described videos shows under the “Non-described videos” section, touching the “Describe” button of any video shall enter the authoring page for that video
	Result	Result is as expected
Case 2	Test Name	Play video with inline description
	Description	Make sure inline audio clip are audible in audio descriptions
	Input	Open the YouDescribe app, from the “Recent Description” list, choose “New Girl 1x02 Kryptonite” video to replay
	Expected output	During the replay, any inline video shall play while the original YouTube video does not pause
	Result	The result is as expected
Case 3	Test Name	Play video with extended description
	Description	Make sure extended audio clips are audible
	Input	Open the YouDescribe app, from the “Recent Description” list, choose “New Girl 1x02 Kryptonite” video to replay
	Expected output	During the replay, any extended video shall play while the original YouTube video shall pause
	Result	The result is as expected
Case 4	Test Name	Switch between different describers
	Description	Change to a different audio description from the default one
	Input	Enter the replay page for video “Friends: The movie (2018)”, choose the author of “Laura Kostler” from the spinner [43] of the “described by”
	Expected output	The audio description provide by “Laura Kostler” is ready to play. Tap the play button shall play the audio description provided by “Laura Kostler”
	Result	The result is as expected
	Test Name	Audio Volume change

Case 5	Description	Increase / decrease the audio volume using the volume control bar or the physical volume control button on the phone while an video with an audio description is in replay.
	Input	Play a video from the recent description list, increase and decrease using the volume control bar or the physical volume control button on the phone
	Expected output	The volume is adjusted accordingly
	Result	The result is as expected
Case 6	Test Name	Log into YouDescribe
	Description	Use google account to log into YouDescribe
	Input	Navigate to the account page and use google account to log in
	Expected output	Log in succeed, with the Portrait of the google account appear on the account page, my description page shall display all the descriptions made by the user
	Result	The result is as expected
Case 7	Test Name	Add a new audio description
	Description	Create a new audio description and record a new audio clip
	Input	Choose video “Dogs, man’s best and funniest friends” from the wish list, tap the “Describe” button, enter the authoring page, add a new audio clip by recording, tap the save button for uploading, then click the publish button on the overflow menu of the toolbar
	Expected output	“Dogs, man’s best and funniest friends” shall appear on the recent description list, with the new audio description and new audio clip added
	Result	The result is as expected
	Test Name	Editing the existing audio description
Case 8	Description	Nudge the starting and point of the an existing audio clip
	Input	Choose video “YELLOW” from my description list and tab “Describe”, Choose the first extended audio clip, nudge the starting point forwardly by 0.5s and choose the type from “extended” to “inline”, then tab “save” button
	Expected output	Go to the search tab and search for “YELLOW” video, choose to replay. The audio clip modified has been moved forward by 0.5s while the it is now play in the “inline” mode, the YouTube video shall not stopp
	Result	The result is as expected
	Test Name	Editing the existing audio description

5.2 Accessibility Testing

We conducted accessibility testing to make sure that the application and its functions work fine for the visually impaired users. Talkback is enabled on our Android devices as introduced in section 4.5. Test case 1 to 5 in the previous section were conducted because accessibilities for these use cases should be ensured for blind or visual impaired people. The basic procedure for accessibility testing is the same as the functional testing except that single tap only triggers the content announcement while users need a double tap to actually select the focused item. The following table gives the details of the accessibility testing for these five cases.

Table 5-2 Accessibility test cases

Case 1	Test Name	Search by Video Name
	Description	Search for videos by using the keyword in the search page
	Input	Navigate to the search tab and enter “yellow” in the search bar
	Expected output	TalkBack speak out the content of the button correctly with users can easily navigate to the button that are desired; Three described videos shows on under the Described Videos section, touching any video shall enter the replay page of that video; Meanwhile 30 non-described videos shows under the “Non-described videos” section, touching the “Describe” button of any video shall enter the authoring page for that video
	Result	The result is as expected
Case 2	Test Name	Play video with inline description
	Description	Make sure inline audio clip are audible in audio descriptions
	Input	Open the YouDescribe app, from the “Recent Description” list, choose “New Girl 1x02 Kryptonite” video to replay
	Expected output	TalkBack speak out the content of the button correctly with users can easily navigate to the button that are desired; During the replay, any inline video shall play while the original YouTube video does not pause
	Result	The result is as expected
Case 3	Test Name	Play video with extended description
	Description	Make sure extended audio clips are audible
	Input	Open the YouDescribe app, from the “Recent Description” list, choose “New Girl 1x02 Kryptonite” video to replay
	Expected output	TalkBack speak out the content of the button correctly with users can easily navigate to the button that are desired; During the replay, any extended video shall play while the original YouTube video shall pause
	Result	The result is as expected
Case 4	Test Name	Switch between different describers
	Description	Change to a different audio description from the default one

	Input	Enter the replay page for video “Yellow”, choose the author of “Cheng Li” from the spinner of the “described by
	Expected output	TalkBack speak out the content of the button correctly with users can easily navigate to the button that are desired; The audio description provide by “Laura Kostler” is ready to play. Tap the play button shall play the audio description provided by “Laura Kostler”
	Result	The result is as expected
Case 5	Test Name	Audio Volume change
	Description	Increase / decrease the audio volume using the volume control bar or the physical volume control button on the phone while a video with an audio description is in replay.
	Input	Play a video from the recent description list, increase and decrease using the volume control bar or the physical volume control button on the phone
	Expected output	TalkBack speak out the content of the button correctly with users can easily navigate to the button that are desired; The volume is adjusted accordingly
	Result	The result is as expected

6 Future Plan

6.1 Improvement based on feedbacks

Software, especially applications on smart phone, should always be developed in accordance to the needs of users. YouDescribe Android app, being a special application that aims to assist the blind and visually impaired people, should take more care of what the visual impaired want and how they feel about it. We provided the beta version of YouDescribe Android app to Dr. Joshua Miele and some other visual impaired for their feedbacks. Our first step of the future plan is to refine YouDescribe Android app according to these feedbacks. Dr. Joshua highly appreciated this application after he reviewed and played with this app. Future refinement plan is listed below.

- A separated text field indicating the author of the current audio description, rather than put that content in the dropdown spinner
- Rating page: change the rating bar to a rating score list
- Rating page: change the list of comments to check box so that the status of choosing can be talk back.
- Add more complicated TalkBack contents to the bottom navigator TabView so that the current chosen tab can be talk back, while status of other tabs can also be spoke out

Besides the visual impaired people, sighted volunteers who contribute audio descriptions are another important portion of users of YouDescribe app. We also sent YouDescribe App to Dr. Yue-Ting Siu who is the director of the volunteer contributor team in Smith-Kettlewell Video Description Research and Development Center. She also gave us lots of instructive suggestions from the perspective of an audio description contributor. The following items are the improvement plans for the authoring part of YouDescribe Android app.

- The recording panel - automatically upload the audio clips as soon as the recording is finished

- The recording panel - Change the "Save" button to "Trash" Button, no need to manually upload, but just provide the feature to delete
- Main UI- Wish List - Describe button: automatically jump to the account tab for user to log in if user is not logged in
- Edit panel along with audio clips list - make the symbol of "Incline / Extended" consistent (Grey for incline and Pink for extended)
- Edit panel: the "save" button is automatically change to "Red" to promote user to remember save as long as something changed.

6.2 Deployment

Another important task is to make YouDescribe Android app public on Google Play Store. There will be a series of operations that need to be considered to have it on public. First, we need to sign our application. Then we need to prepare some store listing materials that are necessary to submit our application to put our application online. Finally, we need to pack our code into the apk package and upload it to the Google Play store. We will follow a detailed official reference provided by Google to finish this process (<https://support.google.com/googleplay/android-developer/answer/113469?hl=en>).

Conclusion

We built the YouDescribe Android app, which provides a novel platform for volunteers to add audio descriptions to YouTube videos to aid the blind and visually impaired users to enjoy the visual content in video clips. In this technical report, we have presented details about the overall architecture, design decisions, development process, and technical aspects in developing accessibility features of an Android application and procedures followed for quality assurance.

This YouDescribe Android application provides a complete, accessible mobile interface to all functionality offered by the YouDescribe website. This includes the ability to search for videos, playback audio descriptions over videos, request audio descriptions, and create new audio descriptions for videos. This app would enable anyone with an Android device to contribute to or enjoy videos on YouDescribe while they are away from their computers. This would not only increase viewership but also increase participation in creating accessible content for everyone.

Reference

1. <https://www.theguardian.com/small-business-network/2014/jan/14/video-content-marketing-media-online>
2. <https://www.marketwatch.com/press-release/4g-lte-market-2018-size-segments-industry-growth-and-global-trends-by-forecast-to-2023-2018-11-01>
3. <http://www.uq.edu.au/teach/video-teach-learn/ped-benefits.html>
4. <http://www.youtube.com>
5. <https://www.hulu.com>
6. https://en.wikipedia.org/wiki/Audio_description
7. Rupal Khilari, YouDescribe iOS app – An accessibility tool for adding audio description to online videos, SFSU master thesis, San Francisco, 2018
8. www.youdescribe.org
9. <https://ski.org>
10. <https://en.wikipedia.org/wiki/Crowdsourcing>
11. <https://9to5mac.com/2016/08/18/android-ios-smartphone-market-share/>
12. https://en.wikipedia.org/wiki/Requirements_analysis
13. [https://en.wikipedia.org/wiki/Android_\(operating_system\)](https://en.wikipedia.org/wiki/Android_(operating_system))
14. <https://en.wikipedia.org/wiki/Node.js>

15. <https://en.wikipedia.org/wiki/MongoDB>
16. <https://en.wikipedia.org/wiki/Mongoose>
17. <https://en.wikipedia.org/wiki/GitHub>
18. [https://en.wikipedia.org/wiki/Java_\(programming_language\)](https://en.wikipedia.org/wiki/Java_(programming_language))
19. https://en.wikipedia.org/wiki/Android_Studio
20. <https://developer.android.com/studio/index.html>
21. <https://developer.android.com/guide/platform/index.html>
22. <https://developers.google.com/youtube/android/player/>
23. https://en.wikipedia.org/wiki/Representational_state_transfer
24. https://en.wikipedia.org/wiki/YouTube_API
25. <https://developer.android.com/guide/topics/ui/accessibility/index.html>
26. <https://support.google.com/accessibility/android/answer/6283677?hl=en>
27. https://support.google.com/accessibility/android/answer/6151827?hl=en&ref_topic=3529932
28. <https://www.smartdraw.com/uml-diagram/>
29. <https://developer.android.com/guide/topics/manifest/manifest-intro.html>
30. <https://developer.android.com/guide/components/services.html>
31. <https://developer.android.com/reference/android/content/Intent.html>

32. <https://developer.android.com/reference/android/content/BroadcastReceiver.html>
33. <https://developer.android.com/guide/topics/ui/declaring-layout.html>
34. <https://developer.android.com/design/downloads/index.html>
35. <https://developer.android.com/guide/topics/ui/menus.html>
36. <https://developer.android.com/samples/BasicMediaRouter/res/values/colors.html>
37. <https://developer.android.com/samples/BasicSyncAdapter/res/values/dimen.html>
38. <https://developer.android.com/guide/topics/resources/more-resources.html#Dimension>
39. <https://developer.android.com/guide/topics/resources/string-resource.html>
40. <https://developer.android.com/guide/topics/ui/layout/listview.html>
41. <https://developer.android.com/reference/android/widget/Adapter.html>
42. <https://developer.android.com/reference/android/widget/Toolbar.html>
43. <https://developer.android.com/guide/topics/ui/controls/spinner.html>
44. <https://developer.android.com/reference/android/widget/Button.html>